

Informe Técnico de Implementación, Pruebas y Calidad de Software

Sistema de Control de Gastos y Presupuestos

Asignatura: Ingeniería de Software.

Carrera: Analista Programador (V).

Institución: Inacap.

Alumna: C. Isabel Flores

Docente: Vedran Tomicic

Fecha: 09, Octubre, 2026

Valdivia, Chile

Índice

INTRODUCCIÓN	4
DESARROLLO	5
Descripción del problema y solución propuesta	5
Propuesta de solución	5
Objetivo general.....	5
Objetivos específicos.....	5
Alcance del prototipo.....	6
Requerimientos del Sistema	6
Requerimientos funcionales	6
Requerimientos No Funcionales.....	7
Trazabilidad de los requerimientos	7
Arquitectura del Sistema.....	7
Diagrama de arquitectura lógica	9
Diseño del sistema mediante diagramas UML	9
Diagrama de casos de uso	10
Diagrama de clases.....	11
Tecnologías utilizadas y justificación técnica	13
Arquitectura y modalidad de despliegue	13
Relación con SaaS, IaaS y Cloud Computing	14
Seguridad del software y análisis OWASP	14
Controles implementados.....	14
Análisis de riesgos OWASP	15
Privacidad, confiabilidad y responsabilidad profesional	16
Plan de pruebas y calidad de software	16
Tipos de pruebas consideradas	16

Matriz de casos de prueba	17
Ejecución de pruebas automatizadas	18
Matriz de la trazabilidad de requerimientos.....	19
Carta Gantt.....	20
Limitaciones y mejoras futuras.....	21
CONCLUSIÓN.....	22
BIBLIOGRAFÍA	23
ANEXOS.....	24

Introducción

La administración de las finanzas personales requiere mantener un registro ordenado de los ingresos y gastos, identificar las principales categorías de consumo y establecer límites que permitan controlar el uso del dinero. Cuando esta información se gestiona manualmente o se encuentra distribuida en diferentes registros, puede resultar más difícil conocer el estado financiero mensual y detectar oportunamente los excesos de gasto.

En respuesta a esta necesidad, se desarrolló Control de Gastos y Presupuestos, una aplicación web que permite registrar ingresos y egresos, organizar movimientos mediante categorías, definir presupuestos mensuales y recibir alertas cuando el gasto alcanza determinados porcentajes del límite establecido.

El presente informe documenta la implementación del software, sus requerimientos funcionales y no funcionales, la arquitectura utilizada, las tecnologías seleccionadas, los mecanismos de seguridad y el proceso de pruebas. Asimismo, contempla la identificación de defectos, las correcciones realizadas y la evaluación del estado final del prototipo, de acuerdo con los criterios de calidad definidos para la evaluación de la unidad.

Desarrollo

Descripción del problema y solución propuesta

El control de las finanzas personales requiere conocer cuánto dinero ingresa, cuánto se gasta y cómo se distribuyen los egresos entre distintas categorías. La falta de un registro centralizado puede dificultar la consulta del historial financiero, el seguimiento de los límites de gasto y la identificación de situaciones en las que un presupuesto está próximo a agotarse.

Por este motivo, se plantea la necesidad de una herramienta que permita centralizar esta información y facilitar su consulta mediante una interfaz accesible desde un navegador web.

Propuesta de solución

La solución consiste en una aplicación web de control financiero personal desarrollada con Python y Django. El sistema permite que cada usuario administre sus propios registros y consulte un resumen mensual de su actividad financiera.

Entre las funcionalidades implementadas en el código se encuentran la gestión de categorías, el registro y mantenimiento de movimientos financieros, la configuración de presupuestos por categoría y mes, la generación de alertas de presupuesto, el filtrado del historial y la exportación de movimientos a formato CSV.

La aplicación utiliza una base de datos SQLite para almacenar la información y mecanismos de autenticación para restringir el acceso a las funcionalidades y a los registros de cada usuario.

Objetivo general

Desarrollar y evaluar una aplicación web para la gestión de ingresos, gastos y presupuestos personales, incorporando funcionalidades de seguimiento financiero, validación de datos, protección de información y pruebas de software que permitan verificar su comportamiento.

Objetivos específicos

- Implementar un mecanismo de registro e inicio de sesión para los usuarios.
- Permitir la creación y administración de categorías financieras.
- Incorporar operaciones para registrar, consultar, modificar y eliminar movimientos.
- Facilitar la configuración y supervisión de presupuestos mensuales por categoría.
- Generar alertas cuando el gasto alcance el 80 % o supere el 100 % del presupuesto definido.
- Presentar un resumen mensual de ingresos, gastos, saldo y distribución de egresos.
- Incorporar validaciones y controles de seguridad para proteger los registros de cada usuario.
- Diseñar y ejecutar pruebas que permitan verificar los requerimientos y documentar los defectos encontrados.

Alcance del prototipo

El alcance contempla la administración de información financiera personal mediante una aplicación web. Incluye autenticación, gestión de categorías, movimientos, presupuestos, alertas, resumen mensual, filtros y exportación de datos.

No se considera implementado el envío de alertas por correo electrónico, la recuperación de contraseñas mediante correo ni un registro de auditoría de los cambios realizados. Estas funcionalidades se identifican como posibles mejoras futuras.

Requerimientos del Sistema

Requerimientos funcionales

Código	Requerimiento Funcional
RF-01	El sistema debe permitir registrar una cuenta de usuario.
RF-02	El sistema debe permitir iniciar y cerrar sesión.
RF-03	El sistema debe permitir crear, consultar, modificar y eliminar categorías propias, respetando las restricciones asociadas a los movimientos existentes.
RF-04	El sistema debe permitir registrar ingresos y gastos con categoría, monto, fecha y descripción.
RF-05	El sistema debe permitir modificar y eliminar movimientos financieros propios.
RF-06	El sistema debe permitir consultar y filtrar movimientos por fechas, categoría, tipo y texto descriptivo.
RF-07	El sistema debe permitir crear, consultar, modificar y eliminar presupuestos mensuales por categoría.
RF-08	El sistema debe generar alertas al alcanzar el 80 % o superar el 100 % del presupuesto configurado.
RF-09	El sistema debe permitir consultar las alertas y marcarlas como leídas.
RF-10	El sistema debe presentar un resumen mensual de ingresos, gastos, saldo y gastos por categoría, además del estado de los presupuestos.
RF-11	El sistema debe permitir exportar los movimientos filtrados a un archivo CSV.

Requerimientos No Funcionales

Código	Requerimiento No Funcional
RNF-01	Seguridad: el sistema debe restringir el acceso a las funcionalidades protegidas mediante autenticación.
RNF-02	Seguridad: el sistema debe restringir el acceso a las funcionalidades protegidas mediante autenticación.
RNF-03	Privacidad: cada usuario debe acceder únicamente a sus propias categorías, movimientos, presupuestos y alertas.
RNF-04	Usabilidad: la interfaz debe facilitar la navegación y el ingreso de información mediante formularios y mensajes de validación.
RNF-05	Mantenibilidad: el código debe separar las vistas, los formularios, los modelos y la lógica de negocio para facilitar su comprensión y modificación.
RNF-06	Mantenibilidad: el código debe separar las vistas, los formularios, los modelos y la lógica de negocio para facilitar su comprensión y modificación.
RNF-07	Protección de sesiones: el sistema debe aplicar controles sobre las cookies y la duración de la sesión.
RNF-08	Calidad: el sistema debe contar con pruebas que permitan verificar sus funcionalidades, validaciones y controles de seguridad.

Trazabilidad de los requerimientos

Los requerimientos funcionales se relacionan con componentes específicos del código: las vistas de `views.py`, las reglas de validación de `forms.py`, las entidades de `models.py` y los servicios de `services.py`.

Esta separación permite identificar qué componente implementa cada requerimiento y facilita la elaboración de casos de prueba asociados a las funcionalidades del sistema.

Arquitectura del Sistema

El sistema Control de Gastos y Presupuestos está desarrollado utilizando Django, un framework web basado en Python que permite organizar la aplicación mediante el patrón arquitectónico Modelo–Plantilla–Vista (MTV). Esta estructura separa la gestión de los datos, el procesamiento de las solicitudes y la presentación de la información, facilitando el mantenimiento y la ampliación del software.

La aplicación se organiza en los siguientes componentes:

- Modelos: definidos en `models.py`, representan las entidades Categoría, Movimiento, Presupuesto y Alerta. Estas clases permiten almacenar y relacionar los datos mediante el ORM de Django.
- Vistas y formularios: las vistas, ubicadas en `views.py`, procesan las solicitudes del usuario y coordinan las operaciones. Los formularios, definidos en `forms.py`, validan los datos antes de que sean procesados.
- Servicios: el módulo `services.py` contiene lógica de negocio relacionada con el cálculo de resúmenes mensuales y la evaluación de presupuestos.
- Plantillas: los archivos HTML presentan la información al usuario mediante una interfaz web que utiliza Bootstrap 5.
- Base de datos: SQLite almacena usuarios, categorías, movimientos financieros, presupuestos y alertas.
- Rutas: los archivos de configuración de URL relacionan las direcciones de la aplicación con las vistas correspondientes.

La separación de estas responsabilidades evita concentrar toda la lógica en un solo componente y favorece que las modificaciones se realicen de manera más controlada.

Diagrama de arquitectura lógica

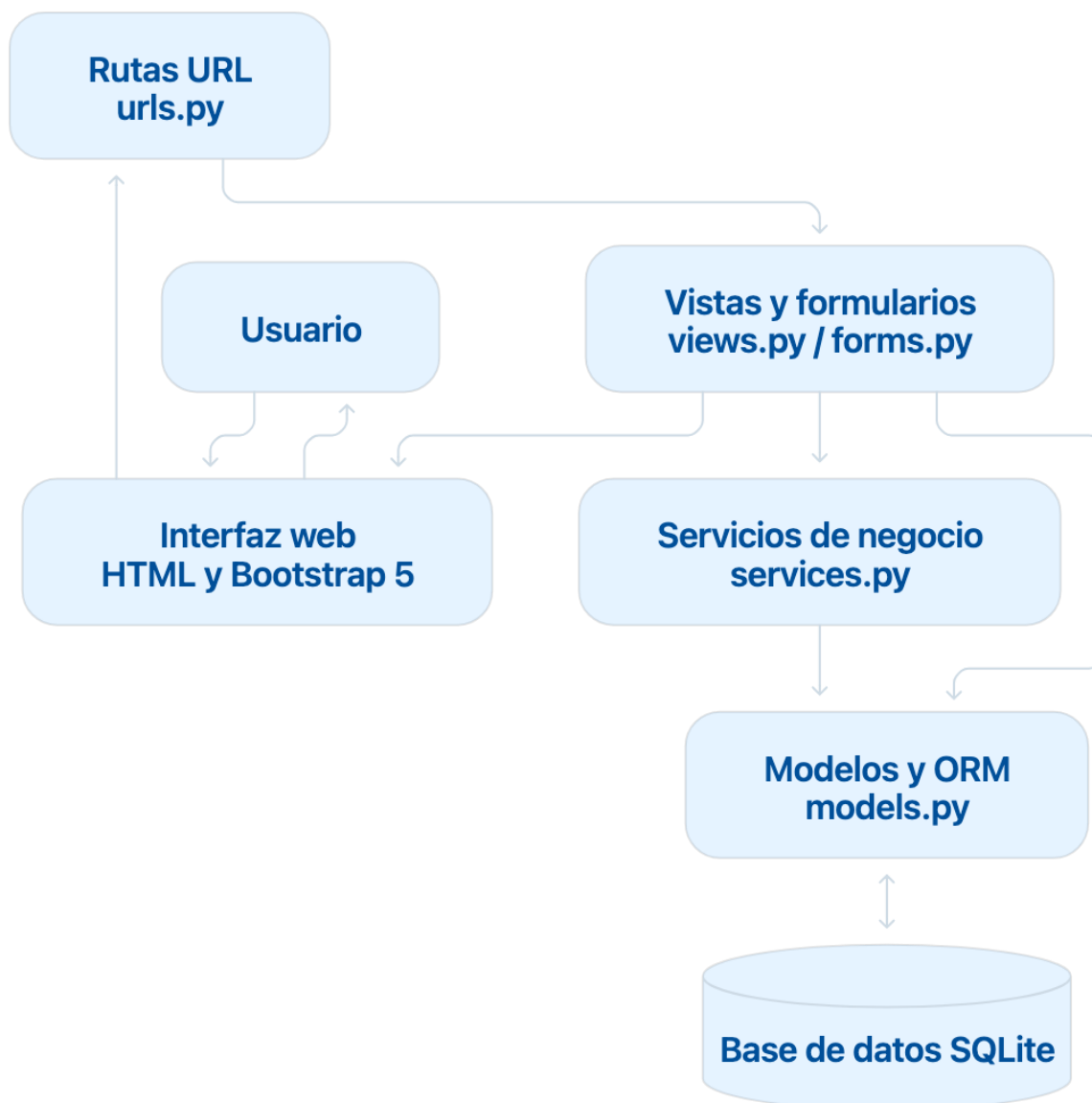


Figura 1. Arquitectura lógica del sistema Control de Gastos y Presupuestos.

El flujo comienza cuando el usuario interactúa con la interfaz web. La solicitud es dirigida mediante las rutas configuradas hacia una vista, que aplica las validaciones necesarias y utiliza los servicios o modelos correspondientes. Cuando se requiere consultar o modificar información, el ORM de Django realiza las operaciones sobre SQLite. Finalmente, la vista entrega la respuesta para su presentación en la interfaz.

Diseño del sistema mediante diagramas UML

Los diagramas UML permiten representar visualmente la estructura y el comportamiento de la aplicación. Para este proyecto se consideran tres diagramas: casos de uso, clases y

actividad. Cada uno cumple una función diferente y debe ser coherente con las funcionalidades identificadas en el código fuente.

Diagrama de casos de uso

El diagrama de casos de uso identifica las acciones que puede realizar una persona dentro del sistema. El actor principal es el usuario registrado, quien puede administrar sus movimientos financieros, categorías, presupuestos y alertas.

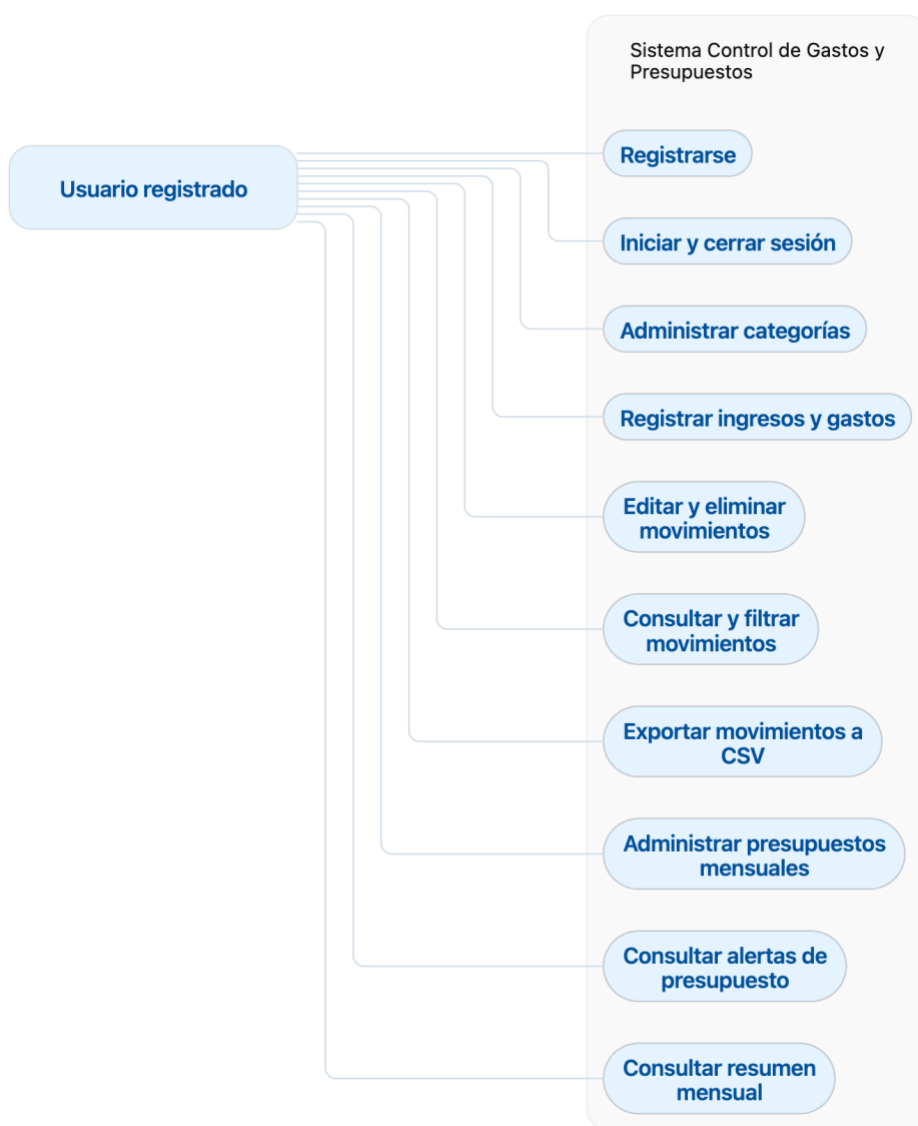


Figura 2. Representación de los casos de uso principales.

El usuario es el único actor funcional representado en este nivel. El registro y la autenticación permiten acceder a las funciones protegidas. Una vez dentro de la aplicación, puede administrar información financiera propia, consultar los resultados de su actividad y exportar los movimientos a un archivo CSV.

La evaluación de los presupuestos y la generación de alertas corresponden a procesos internos del sistema; no requieren un actor externo.

Diagrama de clases

El diagrama de clases representa las principales entidades del modelo de datos y sus relaciones. En la implementación se utilizan cuatro modelos principales de negocio, además del modelo de usuario proporcionado por Django.

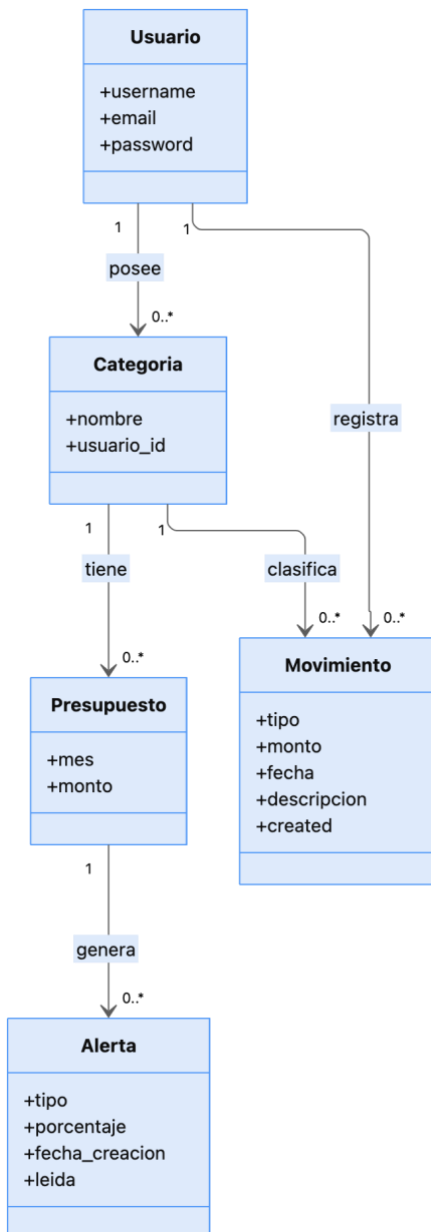


Figura 3. Diagrama de clases del modelo de datos.

La clase Usuario representa las cuentas administradas mediante el sistema de autenticación de Django. Cada usuario puede tener múltiples categorías y movimientos financieros. Las categorías permiten clasificar los movimientos y asociar presupuestos mensuales. Por su

parte, un presupuesto puede estar relacionado con diferentes alertas generadas durante la evaluación de su estado.

La implementación también considera restricciones de integridad. Por ejemplo, no se permite duplicar el nombre de una categoría para un mismo usuario y una misma categoría no puede tener dos presupuestos para el mismo mes. Además, la relación entre categorías y movimientos protege la información histórica al impedir la eliminación de una categoría que todavía posee movimientos asociados.

Diagrama de actividad: evaluación de presupuesto

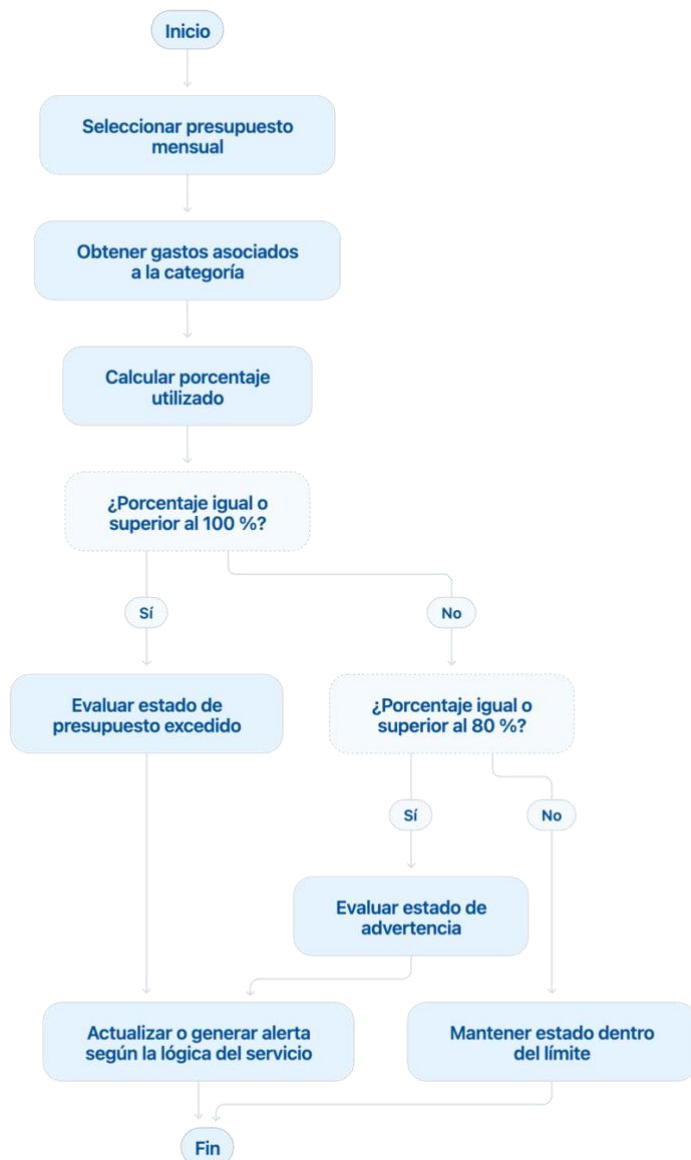


Figura 4. Flujo de evaluación de presupuesto mensual.

Tecnologías utilizadas y justificación técnica

Tecnología	Función dentro del proyecto	Justificación
Python	Lenguaje backend	Permite implementar la lógica del sistema mediante una sintaxis legible y modular.
Django	Framework web	Proporciona autenticación, formularios, enrutamiento, ORM y mecanismos de seguridad.
SQLite	Base de datos relacional	Permite almacenar información estructurada sin necesitar un servidor de base de datos independiente.
HTML	Estructura de las páginas	Define los formularios, tablas y demás elementos de la interfaz.
Bootstrap 5	Presentación visual	Facilita la construcción de una interfaz consistente y adaptable a diferentes tamaños de pantalla.
ORM de Django	Acceso a datos	Permite realizar operaciones sobre la base de datos mediante modelos, reduciendo la necesidad de escribir SQL directamente.
CSV	Exportación de información	Permite descargar movimientos financieros en un formato compatible con hojas de cálculo y otras herramientas.

Arquitectura y modalidad de despliegue

El proyecto utiliza una arquitectura web organizada en componentes, con separación entre presentación, lógica de aplicación y persistencia de datos. Aunque el código incluye una base SQLite local, esto no significa que la aplicación completa sea necesariamente un software exclusivamente offline: Django puede ejecutarse localmente o desplegarse en un servidor.

En su configuración actual, SQLite simplifica el desarrollo y las pruebas al almacenar la información en un archivo. Para una eventual puesta en producción con múltiples usuarios y mayores exigencias operativas, sería necesario evaluar la infraestructura, la concurrencia, las copias de seguridad y la administración de la base de datos.

Relación con SaaS, IaaS y Cloud Computing

Estos conceptos permiten analizar posibles modalidades de distribución del sistema, pero no deben confundirse con tecnologías que ya están implementadas.

- SaaS (Software as a Service): el proyecto podría ofrecerse como un servicio web alojado, donde cada usuario acceda con su cuenta. Para ello sería necesario desplegar y operar la aplicación en un entorno accesible por Internet.
- IaaS (Infrastructure as a Service): podría utilizarse una máquina virtual en la nube para alojar Django y sus componentes de ejecución. Esto requeriría configurar el servidor, la seguridad, la conectividad y el almacenamiento.
- Cloud Computing: representa una alternativa de infraestructura para alojar la aplicación y permitir el acceso remoto. No se identifica en el código revisado una integración específica con un proveedor cloud.

Seguridad del software y análisis OWASP

Debido a que la aplicación administra información financiera personal, la confidencialidad, la integridad y la disponibilidad de los datos son aspectos importantes. Una exposición no autorizada de los movimientos o presupuestos podría afectar la privacidad de los usuarios.

Controles implementados

Riesgo	Control identificado	Evaluación
Acceso a funciones sin autenticación	Protección de las vistas que requieren sesión iniciada	Reduce el acceso no autorizado a las funciones protegidas.
Acceso a información de otra cuenta	Filtrado de consultas según el usuario autenticado	Favorece el aislamiento de los datos entre usuarios.
Entradas inválidas	Validación de montos, fechas, categorías y otros campos	Evita procesar datos que no cumplen las reglas definidas.
Inyección SQL	Uso del ORM de Django en las operaciones de datos	Reduce la necesidad de construir consultas SQL mediante concatenación de cadenas.
XSS	Escape automático de variables en las plantillas de Django	Ayuda a evitar la interpretación de contenido ingresado como código HTML o JavaScript.
Manipulación de archivos CSV	Neutralización de valores que comienzan con caracteres asociados a fórmulas	Reduce el riesgo de que ciertos valores exportados se interpreten como fórmulas en hojas de cálculo.
Robo de sesión mediante lectura de cookies	Cookie de sesión configurada como HttpOnly	Dificulta que JavaScript del navegador lea directamente la cookie.

Acceso a una cuenta por contraseñas débiles	Validadores de contraseña de Django	Aplica controles sobre las contraseñas durante su creación.
Uso de la aplicación en producción sin configuración segura	Configuraciones condicionales para HTTPS, cookies seguras y HSTS	Su efectividad depende de que el entorno de producción esté correctamente configurado.

Análisis de riesgos OWASP

Para una aplicación web como esta, resultan pertinentes varias categorías de riesgo del OWASP Top 10. La tabla presenta una evaluación basada en los controles identificados, no una certificación de ausencia de vulnerabilidades.

Categoría de riesgo	Relación con el proyecto	Medidas y aspectos por revisar
Control de acceso roto	Un usuario podría intentar consultar o modificar movimientos de otra cuenta.	El filtrado por usuario y las respuestas 404 ayudan a restringir el acceso. Debe probarse cada operación protegida.
Fallas criptográficas	Las contraseñas y las sesiones requieren protección.	Django almacena contraseñas mediante algoritmos de hash. Deben verificarse HTTPS y las cookies seguras en producción.
Inyección	Los campos de formularios reciben datos ingresados por el usuario.	El ORM y la validación de formularios reducen el riesgo, pero deben mantenerse las buenas prácticas en futuras modificaciones.
Diseño inseguro	Los errores de lógica pueden producir presupuestos incorrectos o accesos indebidos.	Las reglas de negocio, las restricciones de datos y los casos negativos deben someterse a pruebas.
Configuración de seguridad incorrecta	Una configuración de desarrollo puede no ser adecuada para producción.	Se debe desactivar DEBUG, configurar secretos y hosts permitidos y utilizar HTTPS cuando corresponda.
Fallas de autenticación	La autenticación protege los datos financieros personales.	Se implementa autenticación de Django; el control de intentos fallidos y la recuperación por correo

		aparecen como mejoras pendientes.
Fallas de integridad de software y datos	Una modificación no controlada podría afectar el funcionamiento o los datos.	Se recomienda controlar dependencias, versiones y cambios en el código.
Fallas de registro y monitoreo	Los incidentes pueden pasar inadvertidos sin registros apropiados.	No se debe afirmar que existe una auditoría completa; se recomienda incorporar monitoreo y registro de eventos relevantes.

Privacidad, confiabilidad y responsabilidad profesional

El sistema debe utilizarse de forma responsable, considerando que registra datos relacionados con la vida financiera de las personas. En consecuencia, la aplicación debe limitar el acceso a la información según la cuenta correspondiente, evitar la exposición innecesaria de datos y conservar su integridad.

Desde la perspectiva ética, el desarrollo debe procurar que los usuarios comprendan qué información almacenan y cómo pueden utilizarla. También es importante evitar presentar los resúmenes o alertas como asesoría financiera profesional: son herramientas de organización y seguimiento, no una garantía de que las decisiones económicas tomadas por el usuario sean correctas.

En Chile, la Ley N.º 21.459 sobre delitos informáticos constituye una referencia legal pertinente para el análisis de conductas como el acceso ilícito a sistemas y la afectación de datos o sistemas informáticos. La aplicación de esta normativa debe analizarse según las circunstancias concretas, sin afirmar que cualquier error de software constituye por sí mismo un delito.

Plan de pruebas y calidad de software

El testing permite comprobar que las funcionalidades respondan a los requerimientos, detectar comportamientos incorrectos y verificar que los controles de seguridad funcionen según lo esperado.

El proyecto incluye un archivo finanzas/tests.py con pruebas automatizadas organizadas por áreas funcionales. El código contempla casos relacionados con autenticación, categorías, movimientos financieros y restricciones de acceso, entre otros.

Tipos de pruebas consideradas

- Pruebas funcionales: comprueban el registro, el inicio de sesión, la administración de categorías, el registro de movimientos y la gestión de presupuestos.
- Pruebas negativas: verifican el rechazo de entradas inválidas, como montos negativos, contraseñas incorrectas o datos incompletos.
- Pruebas de autorización: comprueban que un usuario no pueda acceder a los recursos pertenecientes a otra cuenta.

- Pruebas de validación: comprueban restricciones como las fechas permitidas, la longitud de campos y la unicidad de determinadas combinaciones.
- Pruebas de exportación: deben comprobar que el CSV se genere correctamente y que los valores potencialmente peligrosos se neutralicen.

Matriz de casos de prueba

ID	Req.	Funcionalidad evaluada	Datos de entrada	Resultado esperado	Resultado obtenido
CP-01	RF-01	Registro de usuario	Datos válidos y contraseña que cumple las reglas	La cuenta se crea correctamente	La cuenta se crea correctamente
CP-02	RF-02	Inicio de sesión	Usuario válido y contraseña incorrecta	El acceso es rechazado	El acceso es rechazado
CP-03	RF-03	Crear categoría	Nombre válido	La categoría queda asociada al usuario	La categoría queda asociada al usuario
CP-04	RF-04	Registrar gasto	Tipo gasto, monto positivo, fecha válida y categoría propia	El movimiento se guarda correctamente	El movimiento se guarda correctamente
CP-05	RF-04	Validar monto	Monto igual a cero o negativo	El formulario rechaza el dato	El formulario rechaza el dato
CP-06	RF-05	Modificar movimiento	Cambios válidos sobre un movimiento propio	Se actualiza el registro	Se actualiza el registro
CP-07	RF-06	Filtrar movimientos	Rango de fechas válido	Se muestran solo los movimientos que cumplen el filtro	Se muestran solo los movimientos que cumplen el filtro
CP-08	RF-07	Crear presupuesto	Categoría y monto válidos para un mes	El presupuesto se guarda correctamente	Error mes no válido
CP-09	RF-08	Evaluar presupuesto	Gastos que alcanzan el	Se identifica la condición de advertencia	Error mes no válido

			80 % del presupuesto		
CP-10	RF-8	Detectar exceso	Gastos que alcanzan o superan el 100 %	Se identifica la condición de presupuesto excedido	Error mes no válido
CP-11	RF-9	Consultar alertas	Usuario autenticado con alertas propias	Se muestran las alertas correspondientes	Se muestran las alertas correspondientes
CP-12	RF-10	Resumen mensual	Ingresos y gastos registrados en un mes	Se calcula y presenta el resumen correspondiente	Se calcula y presenta el resumen correspondiente
CP-13	RF-11	Exportar CSV	Usuario con movimientos registrados	Se descarga un CSV con los datos permitidos	Se descarga un CSV con los datos permitidos
CP-14	Seguridad	Acceso a recurso ajeno	Identificador de un movimiento de otro usuario	El sistema impide el acceso	El sistema impide el acceso

Ejecución de pruebas automatizadas

Las pruebas se ejecutaron mediante el sistema de pruebas integrado de Django, utilizando el siguiente comando en la terminal:
python manage.py test -v 2

La opción -v 2 permitió visualizar el detalle de cada prueba ejecutada y su resultado. Durante el proceso, Django creó una base de datos temporal para realizar las verificaciones de manera aislada, aplicó las migraciones necesarias y comprobó la configuración general del proyecto.

Resultados obtenidos

La ejecución identificó un total de **96 pruebas automatizadas**, las cuales finalizaron satisfactoriamente. El resultado final fue OK, lo que indica que todas las pruebas ejecutadas fueron aprobadas. Asimismo, el sistema de comprobación de Django no identificó problemas de configuración.

Los resultados se resumen en la siguiente tabla:

Indicador	Resultado
Pruebas identificadas y ejecutadas	96
Pruebas aprobadas	96
Pruebas fallidas	0
Problemas de configuración detectados	0
Tiempo de ejecución	77,378 segundos

Funcionalidades verificadas

Las pruebas se organizaron en diferentes grupos, permitiendo comprobar el comportamiento de las funcionalidades principales del sistema:

- **Autenticación y seguridad:** se verificó el inicio y cierre de sesión, el registro de usuarios, la validación de contraseñas, la protección de rutas privadas y el almacenamiento de contraseñas mediante hash.
- **Gestión de movimientos financieros:** se comprobó el registro, modificación y eliminación de ingresos y gastos, junto con la validación de montos, fechas, tipos de movimiento y categorías asociadas.
- **Gestión de categorías:** se verificó la creación, modificación y eliminación de categorías, incluyendo restricciones para evitar operaciones sobre categorías pertenecientes a otros usuarios.
- **Gestión de presupuestos:** se evaluó la creación, modificación y eliminación de presupuestos, así como la validación de montos, períodos y posibles duplicados.
- **Sistema de alertas:** se comprobó la generación de advertencias al alcanzar el 80 % del presupuesto y de alertas de exceso al llegar o superar el 100 %, además de la prevención de alertas duplicadas.
- **Listado y filtrado de movimientos:** se verificó la búsqueda por texto, tipo, categoría y rango de fechas, así como la paginación y el aislamiento de los datos de cada usuario.
- **Exportación de información:** se comprobó que la exportación respetara los filtros seleccionados y que solo incluyera los movimientos del usuario correspondiente.
- **Resumen financiero:** se verificó la visualización del resumen principal, el cálculo de totales mensuales, los porcentajes de uso del presupuesto y la exclusión de información perteneciente a otros usuarios.

Análisis de los resultados

Los resultados obtenidos demuestran que las funcionalidades cubiertas por las pruebas automatizadas se comportaron conforme a los resultados esperados. La aprobación de los 96 casos permite respaldar el funcionamiento de los módulos evaluados y evidencia la incorporación de controles para validar los datos ingresados, restringir el acceso a información ajena y prevenir operaciones no permitidas.

También se verificaron escenarios negativos, como el ingreso de montos inválidos, el uso de fechas incorrectas, los intentos de acceso a registros de otros usuarios y determinadas entradas potencialmente maliciosas. Estos casos permiten comprobar que el sistema no solo responde ante operaciones válidas, sino que también contempla situaciones que requieren rechazo o protección.

Matriz de la trazabilidad de requerimientos

La trazabilidad relaciona cada requerimiento funcional con los componentes que permiten implementarlo y con el caso de prueba que debe comprobarlo. Esta relación facilita identificar funcionalidades cubiertas y detectar requerimientos que todavía necesitan validación.

Requerimiento	Funcionalidad	Componente principal identificado	Caso de prueba
RF-01	Registro de usuarios	Vistas de autenticación y modelo de usuario	CP-01
RF-02	Inicio y cierre de sesión	Autenticación de Django y rutas protegidas	CP-02
RF-03	Gestión de categorías	Categoría, formularios y vistas de categorías	CP-03
RF-04	Registro de ingresos y gastos	Movimiento, formulario y vista de creación	CP-04, CP-05
RF-05	Modificación y eliminación de movimientos	Vistas de actualización y eliminación	CP-06
RF-06	Consulta y filtrado de movimientos	Vista de listado y lógica de filtrado	CP-07
RF-07	Gestión de presupuestos mensuales	Presupuesto y vistas relacionadas	CP-08
RF-08	Alertas de presupuesto	Servicio de evaluación y modelo Alerta	CP-09, CP-10
RF-09	Consulta de alertas	Vista de alertas y operaciones de lectura	CP-11
RF-10	Resumen mensual	Servicio de resumen y vista principal	CP-12
RF-11	Exportación a CSV	Vista de exportación de movimientos	CP-13

Carta Gantt

Etapas	Actividades principales	Entregable
Análisis	Identificar el problema, los usuarios y los requerimientos	Descripción del problema y requisitos
Diseño	Definir arquitectura, modelos de datos y flujos principales	Diagramas UML y diseño lógico
Implementación	Desarrollar autenticación, categorías, movimientos, presupuestos y alertas	Prototipo funcional
Validación	Revisar las funcionalidades y ejecutar pruebas automatizadas	Matriz de pruebas y resultados
Corrección	Analizar defectos, corregirlos y repetir las pruebas afectadas	Registro de defectos y evidencia

Documentación	Preparar el informe, capturas y anexos	Informe técnico final
---------------	--	-----------------------

Limitaciones y mejoras futuras

Aunque el sistema contiene las funcionalidades principales de administración financiera, existen aspectos que pueden mejorarse antes de considerarlo preparado para un uso real a mayor escala.

Entre las limitaciones identificadas en la documentación del proyecto se encuentran:

- No se identifica un mecanismo de limitación de intentos fallidos de inicio de sesión.
- No se incluye recuperación de contraseña mediante correo electrónico.
- No se dispone de un registro de auditoría completo de las operaciones realizadas.
- Las alertas se consultan dentro de la aplicación y no se envían por correo electrónico.

Como mejoras futuras, se propone incorporar estas capacidades, fortalecer el monitoreo de seguridad, automatizar las copias de respaldo, ampliar las pruebas de integración y validar el comportamiento de la aplicación en un entorno de despliegue real.

También sería conveniente evaluar la base de datos y la infraestructura según el número de usuarios esperado, junto con la configuración de HTTPS, la gestión de secretos y el proceso de actualización de dependencias.

Conclusión

El desarrollo de Control de Gastos y Presupuestos aborda una necesidad concreta: disponer de una herramienta que permita registrar ingresos y gastos, organizarlos por categorías, establecer límites mensuales y consultar información financiera de manera centralizada.

La implementación con Django permite estructurar la aplicación mediante modelos, vistas, formularios y servicios, mientras que SQLite proporciona almacenamiento persistente y Bootstrap 5 facilita la presentación de la información. El proyecto incorpora además controles relevantes para la validación de entradas, el aislamiento de información entre usuarios y la protección de determinadas operaciones.

Sin embargo, la calidad del software no depende únicamente de que las funcionalidades estén programadas. También requiere comprobar su comportamiento mediante pruebas ejecutadas, registrar los defectos detectados y demostrar las correcciones realizadas. Por esta razón, la versión final del informe debe incluir resultados verificables de testing y evidencias correspondientes a la versión entregada.

En conclusión, el proyecto constituye una base funcional para la administración personal de ingresos, gastos y presupuestos. Su fortalecimiento futuro debería centrarse en completar la validación técnica, documentar las correcciones y mejorar las capacidades de seguridad, auditoría y recuperación de cuentas.

Bibliografía

Django Software Foundation. (s. f.). Django documentation.
<https://docs.djangoproject.com/>

OWASP Foundation. (s. f.). OWASP Top 10: Web application security risks.
<https://owasp.org/www-project-top-ten/>

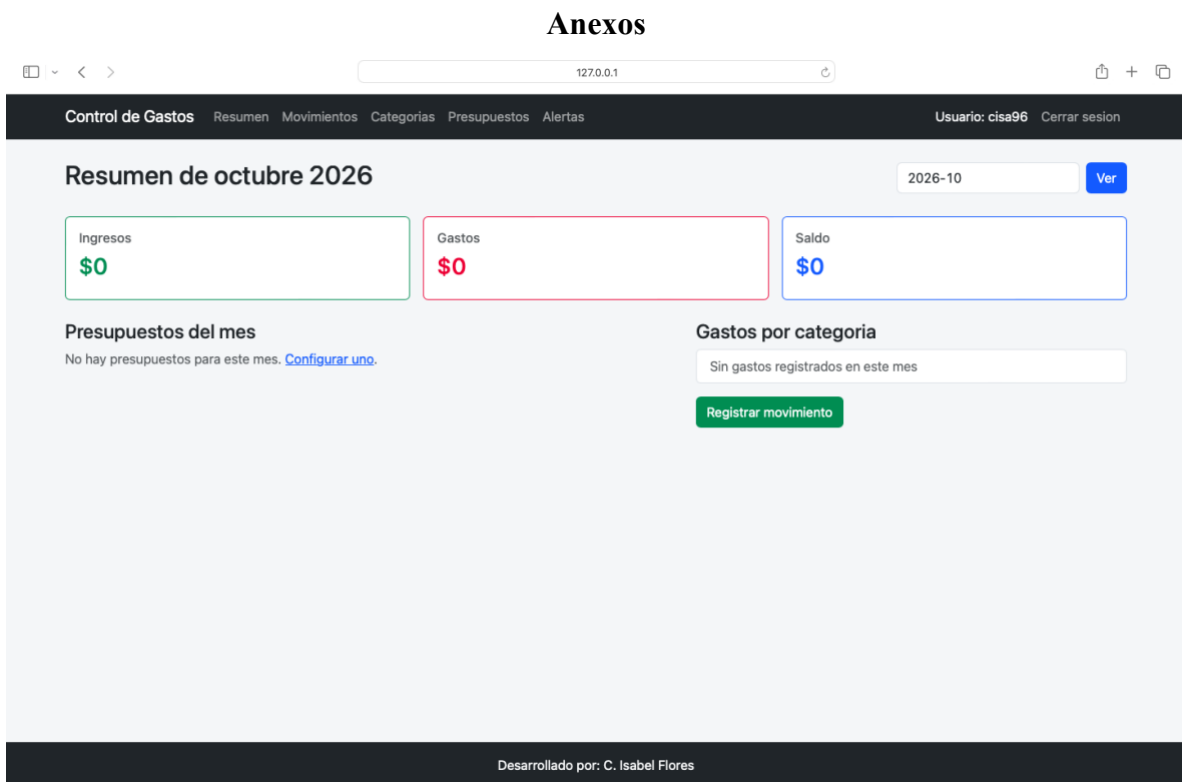
OWASP Foundation. (s. f.). OWASP Cheat Sheet Series.
<https://cheatsheetseries.owasp.org/>

Python Software Foundation. (s. f.). Python documentation.
<https://docs.python.org/3/>

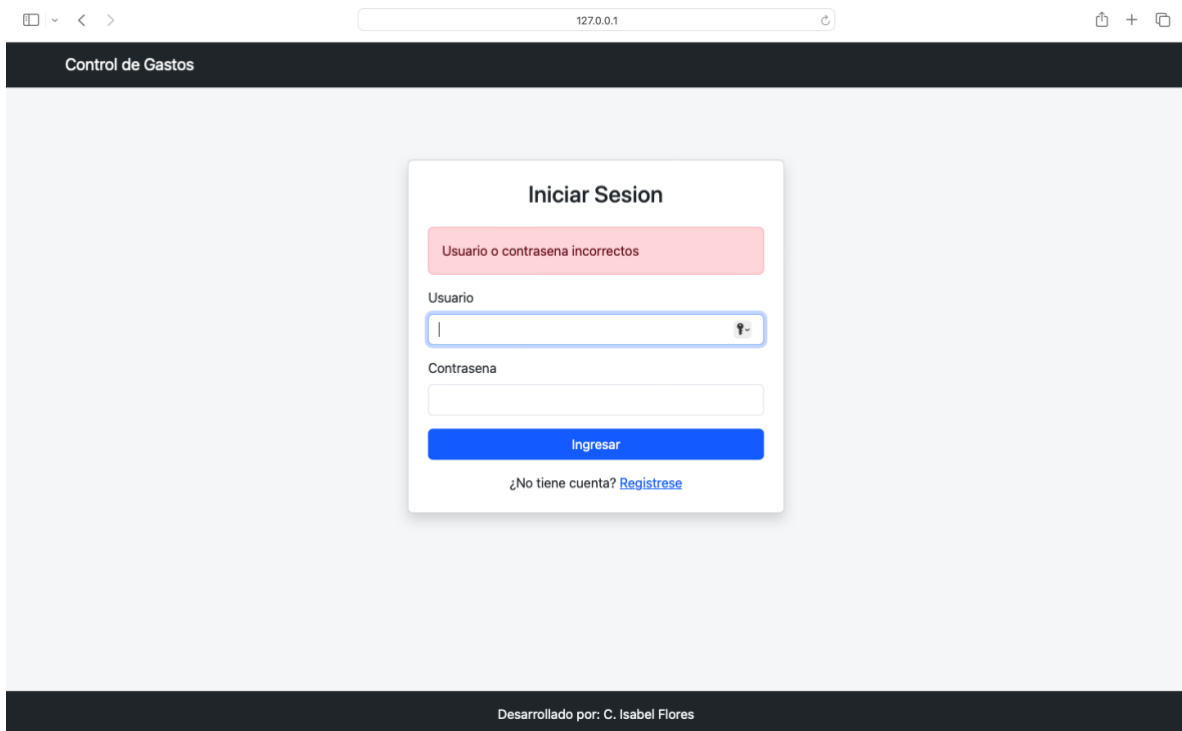
SQLite Consortium. (s. f.). SQLite documentation.
<https://www.sqlite.org/docs.html>

Bootstrap Team. (s. f.). Bootstrap documentation.
<https://getbootstrap.com/docs/>

República de Chile. (2022). Ley N.º 21.459: Establece normas sobre delitos informáticos, deroga la Ley N.º 19.223 y modifica otros cuerpos legales con el objeto de adecuarlos al Convenio de Budapest. Biblioteca del Congreso Nacional de Chile.
<https://www.bcn.cl/leychile/>



CP-01



CP-02

127.0.0.1

Usuario: cisa96 Cerrar sesion

Control de GastosResumenMovimientosCategoríasPresupuestosAlertas

Categoría creada correctamente

Categorías

Nueva categoría

Nombre	Movimientos	Acciones
Alimentacion	0	ModificarEliminar
Entretenimiento	0	ModificarEliminar
Salud	0	ModificarEliminar
Sueldo	0	ModificarEliminar
Transporte	0	ModificarEliminar
Vestimenta	0	ModificarEliminar
Vivienda	0	ModificarEliminar

Desarrollado por: C. Isabel Flores

CP-03

127.0.0.1

Usuario: cisa96 Cerrar sesion

Control de GastosResumenMovimientosCategoríasPresupuestosAlertas

Movimiento registrado correctamente

Movimientos

Exportar CSVRegistrar movimiento

Desde

Hasta

Categoría

Tipo

Texto

09/10/2026

09/10/2026

Todas

Todos

Buscar en descripción

Filtrar

Limpiar

Fecha	Tipo	Categoría	Monto	Descripción	Acciones
09-10-2026	Gasto	Alimentacion	\$48 000	Varios	ModificarEliminar

Desarrollado por: C. Isabel Flores

CP-04

127.0.0.1

Usuario: cisa96 Cerrar sesion

Control de Gastos

Resumen

Movimientos

Categorías

Presupuestos

Alertas

Registrar movimiento

Volver

Monto: Asegúrese de que este valor es mayor o igual a 1.

Tipo

Gasto

Categoría

Salud

Monto

-10000

Fecha

09/10/2026

Descripción

Varios

Guardar

Desarrollado por: C. Isabel Flores

CP-05

127.0.0.1

Usuario: cisa96 Cerrar sesion

Control de Gastos

Resumen

Movimientos

Categorías

Presupuestos

Alertas

Movimiento modificado correctamente

Movimientos

Exportar CSV

Registrar movimiento

Desde

Hasta

Categoría

Tipo

Texto

09/10/2026

09/10/2026

Todas

Todos

Buscar en descripción

Filtrar

Limpiar

Fecha	Tipo	Categoría	Monto	Descripción	Acciones
09-10-2026	Gasto	Alimentacion	\$46 000	Varios	ModificarEliminar

Desarrollado por: C. Isabel Flores

CP-06

127.0.0.1

Usuario: cisa96 Cerrar sesión

Control de Gastos

Resumen

Movimientos

Categorías

Presupuestos

Alertas

Movimientos

Exportar CSV

Registrar movimiento

Desde

Hasta

Categoría

Tipo

Texto

05/10/2026

07/10/2026

Todas

Todos

Buscar en descripción

Filtrar

Limpiar

Fecha	Tipo	Categoría	Monto	Descripción	Acciones
No hay movimientos para mostrar					

Desarrollado por: C. Isabel Flores

CP-07

127.0.0.1

Usuario: cisa96 Cerrar sesión

Control de Gastos

Resumen

Movimientos

Categorías

Presupuestos

Alertas

Nuevo presupuesto

Volver

Mes: Introduzca una fecha válida.

Categoría

Mes

Transporte

Enero

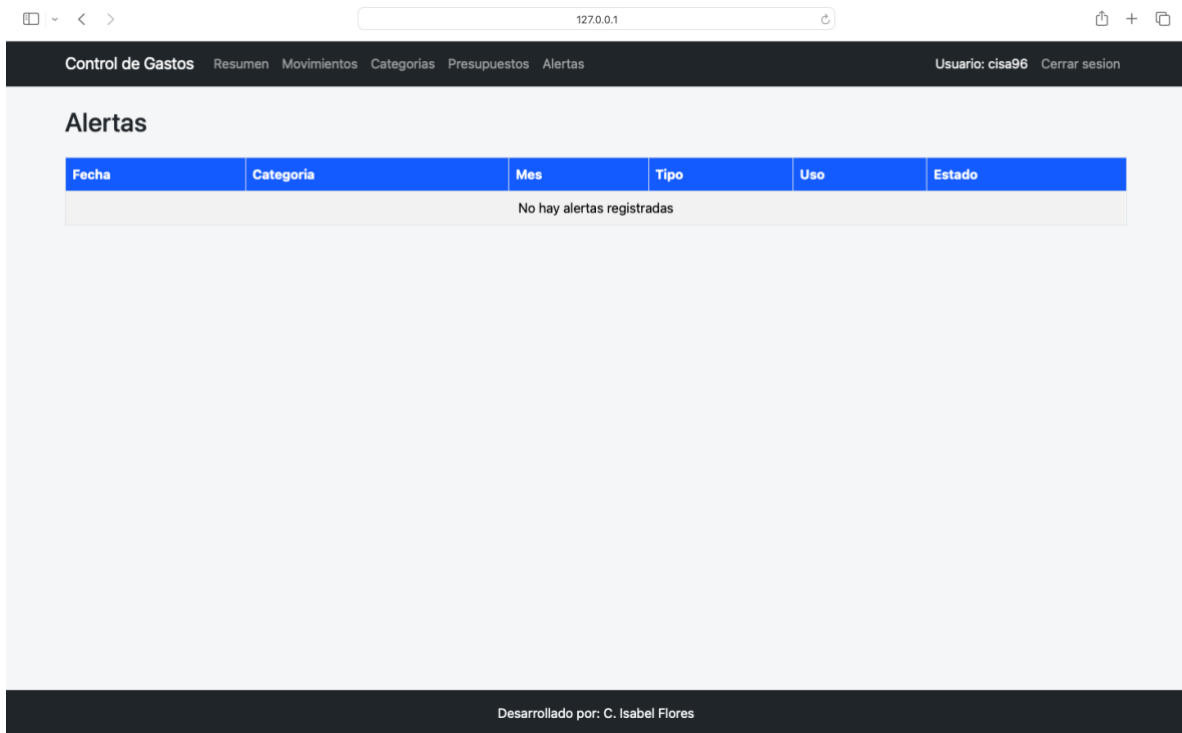
Monto máximo

100000

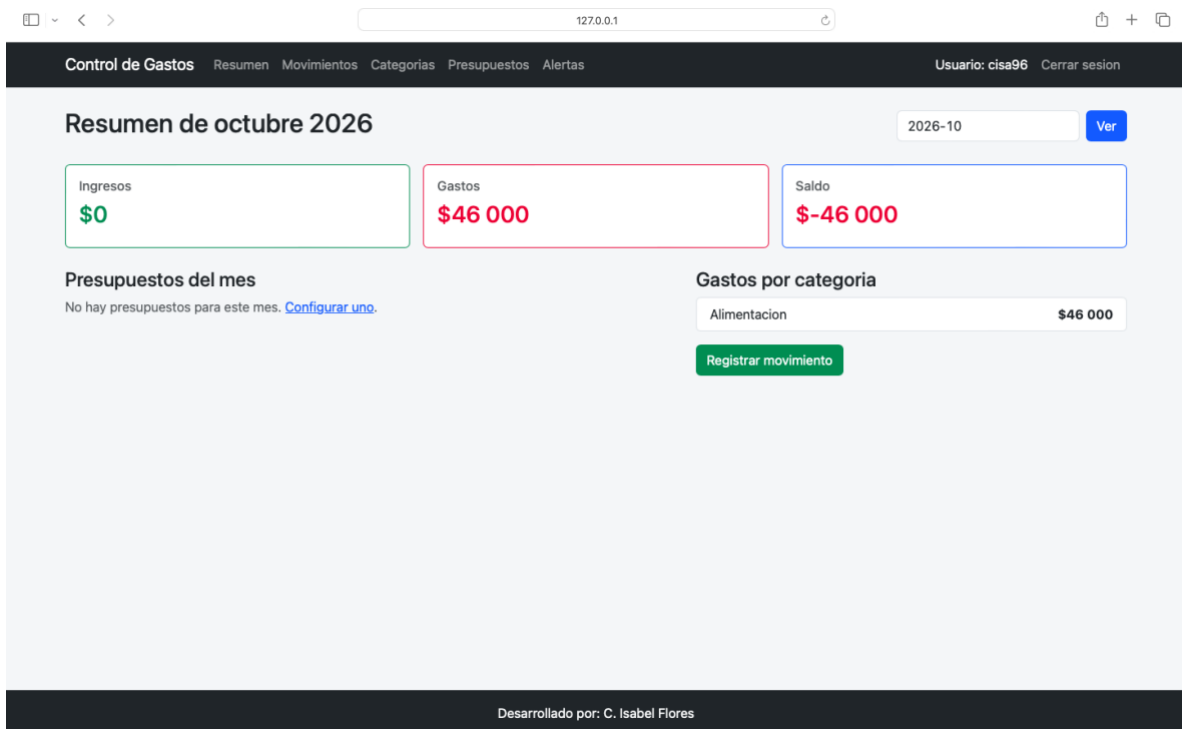
Guardar

Desarrollado por: C. Isabel Flores

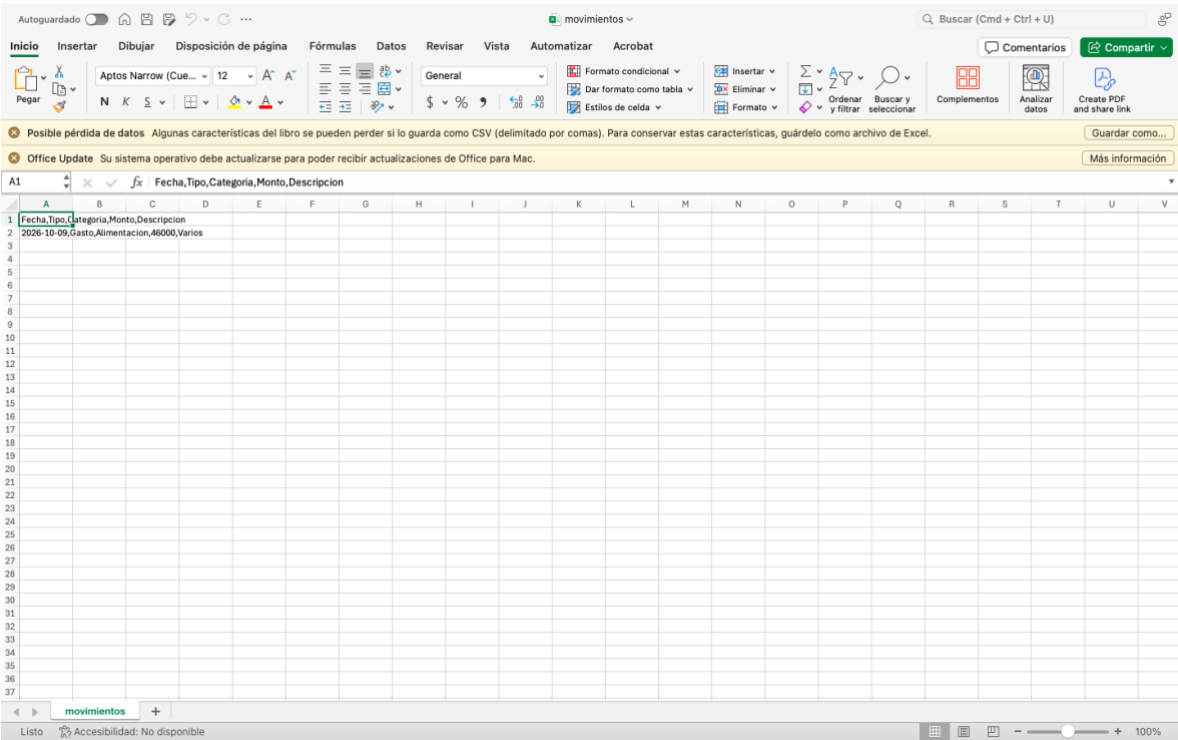
CP-08



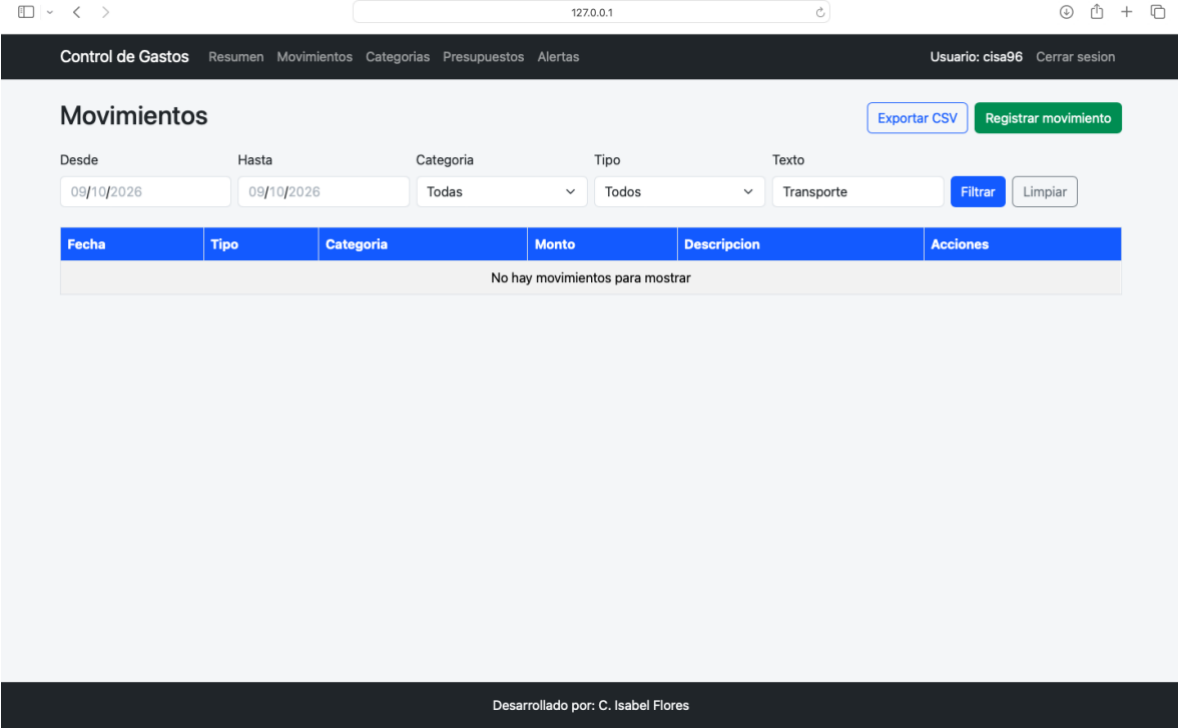
CP-11



CP-12



CP-13



CP-14

